Machine Learning Project: SVM exploration

Anonymous Yuchen Hou submission

Paper ID

1. Introduction

1.1. dataset

The MNIST dataset is a large database of handwritten digits commonly used for training various image processing systems. It consists of 60,000 training images and 10,000 testing images, each of which is a 28x28 pixel gray-scale image. Each image is associated with a label from 0 to 9 representing the digit depicted in the image.

The CIFAR-10 dataset consists of 60,000 32x32 color images in 10 different classes, with 6000 images per class. The classes include airplanes, cars, birds, cats, deer, dogs, frogs, horses, ships, and trucks.

I utilize 5000 image as training data and 1000 images as testing data with balanced labels on both of the MNIST and CIAFR10.

1.2. task

the goal Image classification is to assign a label to an image from a predefined set of categories.

Given an image represented by an n-dimensional feature vector x, and a discrete label y among N classes, the task is to build a classifier that can predict the label of the image based on its features. For SVMs, the problem can be formulated as:

$$\min_{w,b} \sum_{(x,y)\in S} L(y, f_{w,b}(x))$$

where L is a loss function that measures the discrepancy between the predicted label $f_{w,b}(x)$ and the true label y. The SVM aims to find the hyperplane that maximizes the margin between different classes in the feature space.

2. Principle

2.1. SVM

Support Vector Machine (SVM) [1]is a supervised learning model that analyzes data used for classification and regression analysis. The primary goal of SVM is to find a hyperplane in an N-dimensional space that distinctly classifies the data points.

The optimization problem for SVM can be formulated as follows:

$$\min_{\mathbf{w},b} \frac{1}{2}\|\mathbf{w}\|^2 + C\sum_{i=1}^{m} \xi_i \tag{1}$$

subject to

$$y_i(\mathbf{w}\cdot\mathbf{x}_i + b) \geq 1 - \xi_i, \quad \xi_i \leq 0, \forall i \tag{2}$$

where $\mathbf{w}$ is the normal vector to the hyperplane, b is the bias, and $(\mathbf{x}_i, y_i)$ are the training examples with labels.

The Lagrangian of the primal problem is given by:

$$L(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2}\|\mathbf{w}\|^2 - \sum_{i=1}^{N} \alpha_i[y_i(\mathbf{w}\cdot\mathbf{x}_i + b) - 1] \tag{3}$$

The dual problem, obtained by setting the derivatives of L with respect to $\mathbf{w}$ and b to zero, leads to:

$$\max_{\boldsymbol{\alpha}} \sum_{i=1}^{N} \alpha_i - \frac{1}{2}\sum_{i,j=1}^{N} \alpha_i\alpha_j y_i y_j \mathbf{x}_i\cdot\mathbf{x}_j \tag{4}$$

subject to $\alpha_i \geq 0$ and $\sum_{i=1}^{N} \alpha_i y_i = 0$.

2.2. Kernel

The kernel trick in SVM involves mapping the input data into a higher-dimensional space without explicitly performing the computation in that high-dimensional space. This is achieved using a kernel function K, which replace the inner products in optimization problem

$$K(x, z) = \langle\phi(x), \phi(z)\rangle$$

where ϕ is a mapping function that transforms the input vectors x and z into a higher-dimensional feature space.

Kernel functions[2] can vary depending on the specific requirements of the classification task. Below are examples of commonly used kernels:

1. Linear Kernel: The simplest kernel function, which does not actually involve a transformation to a higher-dimensional space. It is defined as:

$$K(x, z) = x^T z$$

2. Polynomial Kernel: The polynomial kernel allows for the learning of polynomial decision boundaries in the original feature space. It is defined as:

$$K(x, z) = (x^T z + c)^d$$

where c is a constant term, and d is the degree of the polynomial.

3. Radial Basis Function (RBF) Kernel:Also known as the Gaussian kernel, the RBF kernel is a popular choice for many classification problems, including image classification. It is defined as:

$$K(x, z) = \exp\left(-\gamma \|x - z\|^2\right)$$

where γ is a parameter that determines the spread of the kernel.

4. Sigmoid Kernel: The sigmoid kernel, similar to the activation function used in neural networks, is defined as:

$$K(x, z) = \tanh(\alpha x^T z + c)$$

where α and c are constants.

2.3. SMO

SMO[3] aims to solve the dual SVM problem by breaking it down into smaller optimization problems, which involve only two Lagrange multipliers at a time. This simplification allows the problem to be solved analytically, leading to faster convergence.

The steps of the SMO algorithm include:

1. Selecting two multipliers, a_i and a_j, that violate the Karush-Kuhn-Tucker (KKT) conditions for optimality.
2. Computing the low and high bounds L and H to ensure that the new multipliers stay within the [0, C] interval after updating.
3. Calculating the second derivative η of the objective function with respect to a_i and a_j, and then updating a_j using:

$$a_j^{new} = a_j^{old} - \frac{y_j(E_i - E_j)}{\eta}$$

where $E_k = \sum_{i=1}^{m} a_i y_i K(x_i, x_k) + b - y_k$ represents the error of prediction on instance x_k.

4. Adjusting a_i based on the update of a_j, and updating the bias term b to reflect the changes in the threshold.

2.4. SVMClassifier

In multi-class classification tasks, Support Vector Machines (SVMs) originally designed for binary classification are extended using the one-against-one (OAO) method.

2.4.1 Training Phase

Each binary classifier is trained on data from two classes. Let (i, j) be a pair of different classes from the set of all classes $[n]$.

To optimize the computational efficiency of the OAO method, classifiers are only trained for each distinct pair, ensuring $i < j$. This reduces the number of classifiers and simplifies the voting process during prediction.

For each pair, the instances belonging only to classes i or j are considered, and the labels are mapped as follows:

$$\sigma_{ij}(y^{(k)}) = \begin{cases} 1 & \text{if } y^{(k)} = i, \\ -1 & \text{if } y^{(k)} = j. \end{cases}$$

The SVM classifier for this pair then learns a model to distinguish these two classes.

2.4.2 Prediction Phase

During prediction, each binary classifier (i, j) votes for one of the two classes it was trained on, based on the sign of the decision function:

$$g_{ij}(x) = \text{sign}(w_{ij}^T x + b_{ij}).$$

The class receiving the most votes across all classifiers determines the predicted class label:

$$h(x) = \arg\max_{i \in [n]} \sum_{j \in [n], j \neq i} g_{ij}(x).$$

3. Experiment

3.1. Linear SVM

My baseline is a linear kernel SVM classifier with SMO optimization and one-to-one voting method. The precision and efficiency of this model in Table 1

The experiment setting for basic model is iterations = 30, penalty(C) = 1

dataset	accuracy	training time	testing time
MNIST	0.90	24.90	5.04
CIFAR10	0.32	102.98	14.67

Table 1. Baseline

3.2. Multi-kernel SVM

I implement various kernels on basic SVM model, the performance of each single kernel is shown in Table 2

kernel	accuracy	training time	testing time
linear	0.904	24.90	**5.04**
Gaussian(rbf)	0.798	316.30	284.46
poly	**0.93**	38.8	9.31
sigmoid	0.681	**19.65**	13.09

Table 2. Multi kernels

The linear kernel achieves the lowest testing time, and relatively high accuracy, suggesting it is the most efficient choice for this dataset.

In contrast, the Gaussian kernel, while offering moderate accuracy, incurs significantly longer training and testing times, indicating a potential trade-off between accuracy and computational efficiency.

The polynomial kernel also performs well in terms of accuracy but is slower than the linear kernel.

The sigmoid kernel shows the lowest accuracy and comparatively higher testing time, making it the least favorable option among the tested kernels.

3.3. Ablation

There are some hyper parameters to optimize in multi-kernel SVM:

1. **Penalty(C)**: I choose C in penalty_list = [0.5 ** i for i in range(0, 8)].The results in shown in Figure 1

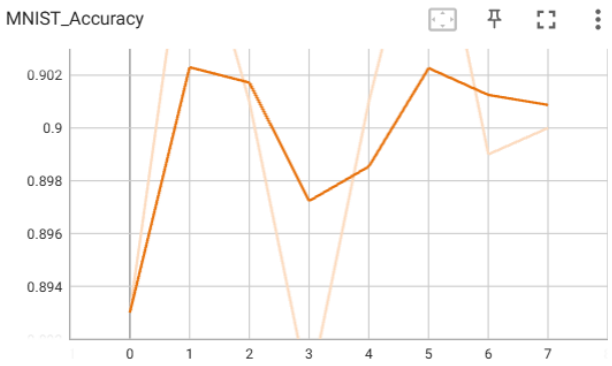

Figure 1. poly kernel

Peak performances are noted at C values approximately equal to 0.5^1 and 0.5^6, suggesting optimal regularization settings for the given model and data.

2. γ **for rbf kernel**: I choose γ in gamma_list = [0.5 ** i for i in range(0, 4)]. The results in shown in Figure 2 We observe a significant improvement in classification accuracy as γ increases from 0.5^0 to 0.5^3, where the accuracy stabilizes above 0.8.

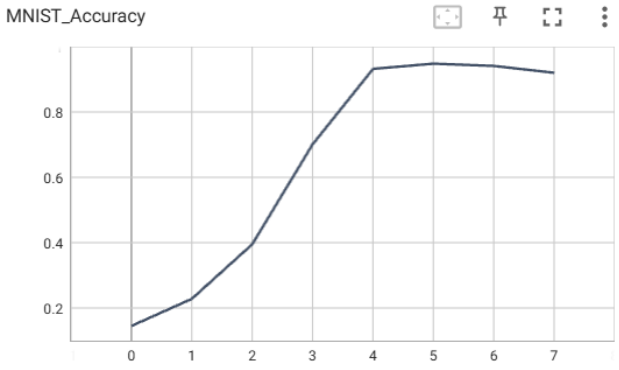

Figure 2. rbf kernel

This performance trend indicates that increasing γ, which narrows the width of the RBF kernel, allows the model to capture more complex patterns in the data, enhancing its ability to differentiate between the digits in MNIST. However, the plateau in accuracy at higher γ values suggests an optimal point beyond which increasing γ does not yield further benefits, possibly due to overfitting to the training data. Further validation could assess whether the accuracy remains stable or decreases, indicating an overfitting scenario.

3. **degree for poly kernel**: I choose degree(d) in degree_list = [i for i in range(1, 5)].The results in shown in Figure 3

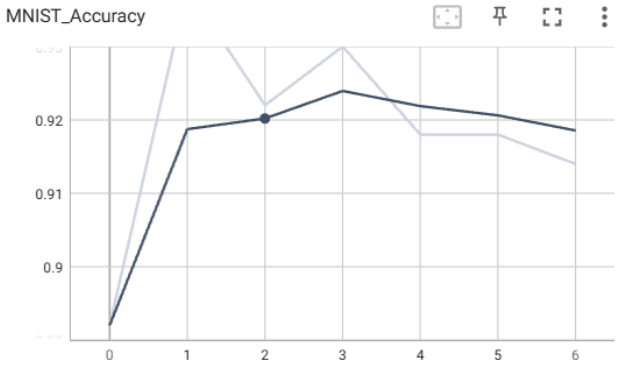

Figure 3. poly kernel

The classification accuracy peaks notably at the third degree polynomial. This suggests that a cubic polynomial kernel might be optimal for this particular SVM configuration and dataset, balancing the model's ability to capture complex patterns without overly fitting the noise in the data.

4. **bias for sigmoid kernel**: I choose bias in bias_list = 0 and γ in γ_list = [0.5 ** i for i in range(0, 8)].The results in shown in Figure 4 γ controls the width of the kernel function, (i.e., the distribution of features in the new mapping space).

The results clearly illustrate a trend where the classification accuracy increases as γ decreases. Initially, the accuracy starts from a lower baseline but shows a consistent increase, moving from below 0.4 to above 0.8 as γ approaches smaller values.

Due to time constraints, I did not try smaller gamma values to get the full results of the ablation experiments on sigmoid kernel. There maybe a over-fitting when γ goes become smaller.

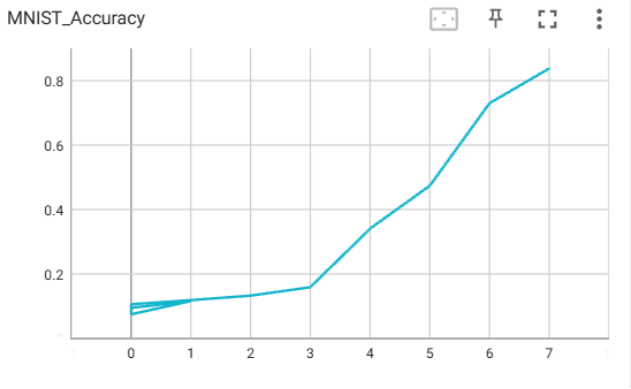

Figure 4. sigmoid kernel

4. Conclusion

This report investigated the performance of Support Vector Machines (SVM) with various kernel functions including linear, Gaussian (RBF), polynomial, and sigmoid on the MNIST and CIFAR10 datasets. Our experiments highlighted distinct differences in accuracy, training time, and testing time across the kernels.

Key findings include:

- The **linear kernel** demonstrated the best overall efficiency, combining high accuracy with low computational demands, making it highly suitable for less complex datasets like MNIST.
- The **polynomial kernel** showed superior accuracy, particularly with a third-degree polynomial, suggesting its usefulness in scenarios requiring the capture of complex patterns, albeit at higher computational costs.
- The **Gaussian (RBF) kernel**, while moderately accurate, proved to be computationally intensive, indicating a trade-off between accuracy and efficiency.
- The **sigmoid kernel** was found to be the least effective, with the lowest accuracy and comparatively high testing times.

I did the ablation studies on different kernel to search for the best hyper-parameters. Adjustments to parameters such as C, γ, and the degree of the polynomial kernel significantly influenced the outcomes.

In conclusion, the choice of kernel and its parameters substantially affects SVM's performance. For practical applications, selecting an appropriate kernel and fine-tuning its parameters according to the dataset's nature can lead to better performance and efficiency.

Due to time constraints, I did not conduct experiments on combined kenel SVM. In the future, I can combine different kernels and automatically assign weights to them according to the degree of match between feature and kernel.

References

[1] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine learning*, 20(3):273–297, 1995. 1

[2] Mehmet Gonen and Ethem Alpaydın. Multiple kernel learning algorithms. *Journal of Machine Learning Research*, 12(Jul): 2211–2268, 2011. 1

[3] John C Platt. Sequential minimal optimization: A fast algorithm for training support vector machines. In *Advances in kernel methods*, pages 185–208. MIT Press, 1998. 2